

IILM UNIVERSITY

Greater Noida

CodSoft

Virtual Internship Program

INTERNSHIP REPORT

Artificial Intelligence

Intern Name	Ayush Tiwari
College	IILM University, Greater Noida
Batch	2CSE13
Roll No.	25SCS1003001616
Internship Domain	Artificial Intelligence
Internship ID	BY26RY226897
Duration	05 August 2026 – 05 September 2026
Organization	CodSoft
Program Manager	Ruman Arshad

Acknowledgment

I would like to express my sincere gratitude to CodSoft for providing me the opportunity to undertake this virtual internship in Artificial Intelligence. This experience has been instrumental in helping me bridge the gap between the theoretical concepts I had studied and their practical, real-world application.

I am especially thankful to Mr. Ruman Arshad, Program Manager at CodSoft, for his guidance, encouragement, and the structured framework provided throughout the internship. The freedom to explore tasks independently, combined with the availability of mentorship and career-support resources such as CodSoft AI, made this a genuinely enriching learning experience.

I would also like to thank my family, friends, and peers for their continuous support and encouragement during the course of this internship. Their motivation played an important role in helping me complete the assigned tasks with sincerity and dedication.

Finally, I am grateful for the wealth of open educational resources, research papers, and developer communities that supported my self-learning throughout this program, and for the opportunity CodSoft has given me to build a practical portfolio in Artificial Intelligence.

— *Ayush Tiwari*

Table of Contents

1. About CodSoft
2. Internship Overview
3. Introduction to Artificial Intelligence
4. Task 1 — Chatbot with Rule-Based Responses
5. Task 2 — Image Captioning
6. Task 3 — Recommendation System
7. Tools and Technologies — Consolidated Summary
8. Challenges Faced and Solutions Adopted
9. Key Learnings and Skill Development
10. Future Scope
11. Conclusion
12. References

1. About CodSoft

CodSoft is a technology education and software development platform built to help aspiring developers turn theoretical knowledge into practical, job-ready skills. It offers virtual internships across domains such as Web Development, Android Development, Python, Data Science, Java, C++, and Artificial Intelligence, combined with project-based learning and AI-driven career tools.

Through its career-support ecosystem, CodSoft provides tools including a resume builder, ATS (Applicant Tracking System) score checker, AI interview coach, career roadmap generator, portfolio builder, and CodSoftAI — a round-the-clock AI mentor for technical guidance and resume review. These tools are intended to accelerate a learner's transition from student to industry-ready professional.

The internship program itself is deliberately self-directed: rather than delivering a fixed curriculum, CodSoft provides a menu of real-world project tasks per domain and expects interns to research, build, and document their own solutions. This design mirrors the ambiguity and self-driven problem-solving expected in real software engineering roles, and is reinforced by the requirement that interns maintain a public GitHub repository and publish video walkthroughs of their work on LinkedIn — building both a technical portfolio and a professional network in parallel.

2. Internship Overview

This report documents the work completed as part of the CodSoft Artificial Intelligence virtual internship.

- **Intern:** Ayush Tiwari
- **Internship ID:** BY26RY226897
- **Domain:** Artificial Intelligence
- **Duration:** 05 August 2026 to 05 September 2026 (1 month)
- **Mode:** Remote / Virtual
- **Minimum requirement:** At least 3 of 5 prescribed tasks, submitted with source code on a dedicated GitHub repository and a demonstration video posted on LinkedIn with the #codsoft hashtag

Of the five available tasks — a rule-based chatbot, a Tic-Tac-Toe AI, image captioning, a recommendation system, and face detection and recognition — the following three were completed for this internship and are detailed in this report:

- Task 1: Chatbot with Rule-Based Responses
- Task 3: Image Captioning
- Task 4: Recommendation System

Each task was approached as an independent mini-project: understanding the underlying concept, researching existing approaches and reference implementations, designing and building a working solution, and reflecting on its limitations. The following sections present each task in detail, followed by a consolidated view of the tools used and the challenges encountered across the internship as a whole.

3. Introduction to Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science concerned with building systems that perform tasks which typically require human intelligence, such as understanding language, recognizing patterns, making decisions, and learning from experience. AI is generally studied across a spectrum of techniques: purely symbolic, rule-based systems at one end, where behaviour is explicitly programmed by a developer, and data-driven statistical machine learning and deep learning systems at the other, where behaviour emerges from patterns learned automatically from data.

Between these extremes sit classical machine learning techniques — such as similarity search, clustering, and matrix-based methods — which learn simple statistical relationships from data without the heavy computation of deep neural networks. Deep learning, a further specialisation of machine learning based on multi-layered neural networks, has in recent years become the dominant approach for tasks involving unstructured data such as images, audio, and natural language, largely because it can automatically learn useful feature representations rather than requiring them to be hand-engineered.

The three tasks undertaken in this internship were deliberately chosen to span this spectrum, giving a broad, practical grounding across the discipline:

- **Task 1 (Chatbot):** an example of symbolic, rule-based AI, where intelligence comes from explicit, hand-written logic rather than learned parameters.
- **Task 4 (Recommendation System):** an example of classical machine learning, using similarity and interaction data rather than deep neural networks.
- **Task 3 (Image Captioning):** an example of deep learning, combining computer vision and natural language processing in a single end-to-end trainable system.

4. Task 1 — Chatbot with Rule-Based Responses

4.1 Objective

To build a chatbot that responds to user input using predefined rules rather than machine learning, providing a foundational understanding of natural language processing and conversational flow, and to appreciate why this architecture was the starting point from which modern conversational AI evolved.

4.2 Background and Research

Rule-based chatbots represent the earliest and simplest architecture for conversational agents. Rather than learning from data, they compare user input against a fixed set of patterns and return a pre-written response when a match is found.

The approach traces back to ELIZA, built in 1966, which simulated a psychotherapist by matching keywords in user sentences to transformation rules and reflecting the input back as a question. Its successor, ALICE, refined this pattern-and-template approach and popularised AIML (Artificial Intelligence Markup Language) as a standard way to encode such rules. Later systems such as ChatScript extended this idea with additional NLP pre-processing, including part-of-speech tagging and synonym resolution, to improve matching accuracy without introducing machine learning.

Two implementation techniques are commonly combined in rule-based bots:

- **Pattern matching:** the user's message is compared against a library of patterns, often using regular expressions or keyword rules, to identify the closest matching rule and trigger its associated response.
- **Parsing and NLP pre-processing:** libraries such as NLTK or spaCy are used for tokenisation and named-entity recognition, allowing the bot to extract entities (such as names, dates, or locations) and produce slightly more flexible, context-aware responses even though the underlying logic is still rule-driven.

This design is easy to build, fully predictable, and requires no training data, which makes it well suited to narrow, well-defined domains such as FAQs or simple customer support. Its principal limitation is scalability: covering a broader range of conversation requires manually writing an increasing number of rules, and the bot cannot generalise to phrasing it has not been explicitly programmed to recognise. This is precisely why the field later moved toward retrieval-based and, eventually, generative machine-learning-driven chatbots capable of handling open-ended conversation.

4.3 Methodology

The chatbot developed for this task follows a standard rule-based design:

- **Input handling:** user text is normalised (lower-cased, punctuation stripped, extra whitespace removed) before matching.
- **Intent rules:** a dictionary/regex-based rule set maps common intents — greetings, farewells, small talk, and domain-specific queries — to one or more candidate responses.
- **Response selection:** when an input matches a rule, a response is chosen (optionally at random among several phrasings for variety); unmatched input falls back to a generic clarifying response rather than failing silently.
- **Conversation loop:** the bot runs in a loop, reading input and producing output, maintaining simple state (e.g., the user's name) across turns where needed.

4.4 Tools and Technologies Used

Tool / Technology	Role in the Project
Python	Core programming language used to implement the chatbot logic.
re (Regex module)	Used for pattern matching between user input and predefined rules.
NLTK	Used for tokenisation and (optionally) its built-in Chat utility for reflection-based responses.
spaCy (optional)	Used for lightweight named-entity recognition to extract entities such as names or dates.

4.5 Challenges Faced

- ▶ Handling variety in phrasing: users express the same intent in many different ways, and a purely keyword-based system initially failed to recognise valid but unanticipated phrasings — addressed by broadening regex patterns and adding synonym lists.
- ▶ Avoiding rigid, repetitive responses: returning the same fixed reply for a given rule felt unnatural — addressed by maintaining multiple candidate responses per rule and selecting among them at random.
- ▶ Rule conflicts and ordering: some user inputs matched more than one rule, requiring careful ordering of rules from most specific to most general so the correct one fires first (the same specific-to-general ranking used in ELIZA's original design).
- ▶ Graceful failure: ensuring the bot never crashes or goes silent on unrecognised input, but instead falls back to a sensible clarifying response.

4.6 Learning Outcomes

This task built a practical understanding of intent recognition, the trade-off between rule coverage and maintainability, and the groundwork on which more advanced retrieval-based and generative chatbots are built.

5. Task 2 — Image Captioning

5.1 Objective

To combine computer vision and natural language processing into a single system that generates a natural-language description of the contents of an image.

5.2 Background and Research

Image captioning is typically framed as an encoder–decoder problem, a design borrowed from machine translation. The encoder converts the image into a compact numerical representation, and the decoder converts that representation into a sequence of words.

In the classical CNN–RNN architecture, a Convolutional Neural Network — commonly a pre-trained model such as VGG, ResNet, or InceptionV3 — acts as the encoder, extracting a fixed-length feature vector that summarises the image's visual content. This feature vector is then fed into a Recurrent Neural Network decoder, typically built from LSTM or GRU units, which is trained to predict one word at a time, conditioned on the image features and the words generated so far. During training, "teacher forcing" is used, feeding the ground-truth previous word into the decoder; during inference, the model instead feeds back its own previous prediction, which can compound small errors over a sentence.

A significant refinement of this architecture is the addition of an attention mechanism, which allows the decoder to focus on different regions of the image as it generates each word, rather than compressing the entire image into a single static vector. More recently, Transformer-based architectures have replaced RNN decoders altogether: because self-attention processes all positions in parallel rather than sequentially, Transformers train faster and better capture long-range dependencies between image regions and generated words, and are now considered state of the art for this task.

Captioning models are commonly trained and benchmarked on datasets such as MS COCO, Flickr8k, and Flickr30k, and evaluated using metrics like BLEU, METEOR, and CIDEr, which quantify overlap between generated and human-written reference captions.

5.3 Methodology

- Feature extraction: a pre-trained CNN (e.g., VGG16/ResNet50/InceptionV3), with its final classification layer removed, is used to extract an image feature vector.
- Text preparation: caption data is cleaned and tokenised, a vocabulary is built from the training captions, and each word is mapped to an embedding for use by the decoder.
- Sequence generation: the image feature vector is passed into an LSTM/GRU decoder (or a Transformer decoder in the more advanced variant), which is trained to predict the next word in the caption at each time step.
- Inference: at prediction time, the decoder generates words one at a time, feeding each predicted word back in as input, until it produces an end-of-sequence token or reaches a maximum caption length.
- Evaluation: generated captions are compared against held-out reference captions using BLEU scores to gauge fluency and relevance.

5.4 Tools and Technologies Used

Tool / Technology	Role in the Project
Python	Core programming language for data processing and model implementation.
TensorFlow / Keras or PyTorch	Deep learning framework used to build and train the encoder–decoder model.
Pre-trained CNN (VGG16 / ResNet50 / InceptionV3)	Used as a fixed feature extractor via transfer learning, avoiding the need to train a vision model from scratch.
LSTM / GRU (or Transformer decoder)	Sequence model used to generate captions word by word from the image features.
Flickr8k / Flickr30k dataset	Publicly available image–caption dataset used for training and evaluation.
NumPy / Matplotlib	Used for data handling and visualising sample predictions.

5.5 Challenges Faced

- ▶ High computational cost: training an encoder–decoder model from scratch is resource-intensive — mitigated by using transfer learning with a pre-trained, frozen CNN encoder rather than training the vision component from scratch.
- ▶ Limited and repetitive vocabulary: with a modest dataset such as Flickr8k, the model tends to default to generic, high-frequency phrases rather than precise descriptions — a well-documented limitation of encoder–decoder captioning models trained on smaller datasets.
- ▶ Exposure bias between training and inference: because training uses teacher forcing (feeding ground-truth words) while inference feeds back the model's own predictions, small early errors can cascade into an increasingly inaccurate caption — a known limitation of the classic CNN-RNN design.
- ▶ Evaluating caption quality: automatic metrics like BLEU do not always correlate well with human judgement of caption quality, making it necessary to manually inspect a sample of generated captions alongside the quantitative score.

5.6 Learning Outcomes

This task provided hands-on experience with transfer learning, sequence generation, and multimodal systems that bridge computer vision and NLP — core concepts behind many real-world applications such as accessibility tools for visually impaired users and automated image indexing.

6. Task 3 — Recommendation System

6.1 Objective

To build a system that suggests relevant items — such as movies, books, or products — to a user based on their preferences or the behaviour of similar users.

6.2 Background and Research

Recommender systems are generally built using one of two core approaches, or a hybrid of both.

- **Content-based filtering:** recommends items similar to ones a user has liked before, based on item attributes (e.g., genre, director, and cast for movies, or category and description for products). It relies on comparing item feature vectors — often built with techniques like TF-IDF — and does not require data from other users, but its recommendations tend to stay within a user's existing taste and depend heavily on well-engineered item metadata.
- **Collaborative filtering:** recommends items based on the preferences of other, similar users, using only the user-item interaction (ratings) matrix rather than item content. It has two common forms: user-based, which finds users with similar rating patterns and recommends what they liked, and item-based, which finds items that tend to be rated similarly by the same users. Collaborative filtering can surface unexpected recommendations outside a user's usual taste, but suffers from the 'cold-start' problem for new users or items with little interaction history, and from data sparsity in large catalogues.

Because each approach has complementary strengths, many production systems — including those used by large platforms — combine both into hybrid recommenders, merging collaborative behavioural signals with content-based item features, sometimes through weighted averaging or a learned ranking layer, to produce more robust recommendations than either method alone.

6.3 Methodology

- **Data preparation:** a user-item interaction dataset (ratings or preferences) and/or item metadata (genre, tags, description) is collected and cleaned.
- **Similarity computation:** for collaborative filtering, similarity between users or items is computed (e.g., cosine similarity or Pearson correlation) from the ratings matrix; for content-based filtering, similarity is computed between item feature vectors built with TF-IDF.
- **Recommendation generation:** for a target user, the system ranks candidate items by predicted preference — either the preferences of the nearest similar users (collaborative) or similarity to items the user already liked (content-based) — and returns the top-N results.
- **Evaluation:** recommendation quality is assessed using metrics such as precision@k, recall@k, or root-mean-square error (RMSE) on held-out ratings.

6.4 Tools and Technologies Used

Tool / Technology	Role in the Project
Python	Core programming language for the recommendation pipeline.
pandas / NumPy	Data loading, cleaning, and manipulation of the ratings and metadata.
scikit-learn	Cosine similarity computation and TF-IDF vectorisation for content-based filtering.
MovieLens dataset	Publicly available ratings dataset used for building and testing collaborative

Tool / Technology	Role in the Project
	filtering.
Matplotlib / Seaborn	Visualising rating distributions and evaluating recommendation quality.

6.5 Challenges Faced

- ▶ Cold-start problem: new users with no rating history, or new items with no interactions, could not be meaningfully recommended using collaborative filtering alone — addressed by falling back to content-based or popularity-based recommendations for such cases.
- ▶ Data sparsity: with many users and items, the user–item ratings matrix is overwhelmingly empty, which weakens similarity calculations — addressed by restricting similarity computation to users/items with a minimum number of shared ratings.
- ▶ Scalability of similarity computation: computing pairwise similarity across all users or items grows expensive as the dataset grows — addressed by limiting the algorithm to a bounded neighbourhood (top-k similar users/items) rather than a full pairwise comparison.
- ▶ Over-specialisation in content-based filtering: content-based recommendations tended to stay narrowly within a user's already-known preferences, reducing discovery of new types of content — a well-documented limitation of pure content-based methods.

6.6 Learning Outcomes

This task developed practical understanding of similarity metrics, the collaborative-versus-content-based trade-off, and the cold-start and sparsity challenges that shape real-world recommender system design.

7. Tools and Technologies — Consolidated Summary

The table below consolidates the primary tools and technologies used across all three tasks completed during this internship.

Tool / Technology	Role in the Project
Python	Primary programming language used across all three tasks.
re / NLTK / spaCy	Pattern matching and NLP pre-processing for the rule-based chatbot.
TensorFlow / Keras / PyTorch	Deep learning framework used to build the image captioning model.
Pre-trained CNNs (VGG16 / ResNet50 / InceptionV3)	Transfer learning for image feature extraction in the captioning task.
LSTM / GRU / Transformer	Sequence generation for producing image captions.
pandas / NumPy / scikit-learn	Data handling, similarity computation, and vectorisation for the recommendation system.
Public datasets (Flickr8k, MovieLens)	Training and evaluation data for image captioning and recommendation tasks respectively.
Jupyter Notebook / VS Code	Development environment used for iterative experimentation and debugging.
GitHub	Version control and public hosting of all task source code, as required for internship submission.
LinkedIn	Publishing project demonstration videos as proof of work, as required for internship submission.

8. Challenges Faced and Solutions Adopted

Beyond the task-specific challenges detailed in earlier sections, several challenges recurred across the internship as a whole:

- **Self-directed learning curve:** unlike a structured course, this internship provided only a task description and expected independent research and implementation. This required consulting official documentation, research papers, and community tutorials to fill gaps in understanding — closely mirroring the self-driven problem-solving expected in professional software roles.
- **Environment and dependency setup:** configuring compatible versions of deep learning libraries, pre-trained models, and datasets occasionally caused version-conflict errors — resolved by using isolated virtual environments and pinning dependency versions.
- **Balancing model complexity with available compute:** deep learning tasks such as image captioning are computationally demanding; working within limited local compute meant scaling down dataset size, image resolution, or training epochs to keep experiments tractable.
- **Debugging opaque failures:** unlike simple logic bugs, incorrect outputs from a trained model (e.g., a chatbot rule silently misfiring, or a captioning model producing a nonsensical sentence) required systematic testing with small, controlled inputs to isolate the cause.

- **Documentation and reproducibility:** maintaining clean, well-commented code and a clear GitHub repository structure (as required by the CODSOFT_TASKSNO naming convention) required additional discipline beyond simply getting the code to work.

9. Key Learnings and Skill Development

Across the three completed tasks, this internship strengthened both technical and professional skills:

- A working understanding of three distinct AI paradigms: rule-based symbolic systems, classical machine learning via similarity search, and deep learning for multimodal perception and generation.
- Practical experience with the Python AI/ML ecosystem, including NLTK, TensorFlow/Keras or PyTorch, scikit-learn, and pandas.
- The ability to source, clean, and prepare real-world datasets (Flickr8k, MovieLens) for model training and evaluation.
- Improved debugging and experimentation discipline when working with non-deterministic, data-driven systems.
- Version control practices using Git and GitHub, and the ability to present technical work clearly through a recorded demonstration.
- A stronger appreciation of the trade-offs — accuracy, interpretability, data requirements, and compute cost — that guide real-world AI system design choices.

10. Future Scope

Each of the three implementations completed during this internship can be extended further:

- **Chatbot:** the rule-based system could be extended with a machine-learning-based intent classifier, or replaced entirely with a retrieval-based or transformer-based generative model, to handle a wider range of unanticipated user inputs.
- **Image Captioning:** the current CNN–RNN pipeline could be upgraded to an attention-based or fully Transformer-based architecture, and trained on a larger dataset such as MS COCO, to improve caption diversity and accuracy.
- **Recommendation System:** the current implementation could be extended into a hybrid recommender that combines collaborative and content-based signals, along with contextual features such as time of use, to improve both recommendation relevance and cold-start handling.

11. Conclusion

Across these three tasks, the internship provided exposure to three distinct paradigms within Artificial Intelligence: symbolic, rule-driven reasoning (the chatbot); classical machine learning based on similarity and interaction data (the recommendation system); and deep learning for multimodal perception and generation (image captioning). Together, they offered practical, project-based experience translating AI concepts — pattern matching, similarity search, transfer learning, and sequence generation — into working implementations, in line with CodSoft's project-based approach to learning.

This internship, completed under CodSoft's Artificial Intelligence program from 05 August 2026 to 05 September 2026, satisfied the minimum requirement of three completed tasks, supported by source code

maintained on GitHub and project walkthroughs shared on LinkedIn, as required for successful completion. Beyond the technical outcomes, the experience of researching, building, documenting, and presenting independent AI projects has been a valuable step toward professional readiness in the field.

12. References

1. ScienceDirect — "Chatbots: History, technology, and applications" (rule-based chatbot architecture, ELIZA, ALICE, AIML).
2. ScienceDirect — "A survey on chatbots and large language models: Testing and evaluation techniques" (pattern matching and NLP components in chatbot design).
3. GeeksforGeeks — "Building a Rule-Based Chatbot with Natural Language Processing."
4. arXiv 1903.09025 — "Recent advances in conversational NLP: Towards the standardization of Chatbot building" (rule ordering and specific-to-general keyword ranking).
5. Medium (Subash Palvel) — "Image Captioning using CNN-RNN Architectures" (encoder-decoder design, LSTM/GRU decoding, teacher forcing).
6. arXiv 2204.07374 — "Image Captioning In the Transformer Age" (CNN-RNN heterogeneous architecture and its limitations).
7. arXiv 2509.17365 — "Pre-Trained CNN Architecture for Transformer-Based Image Caption Generation Model" (Transformer decoders replacing RNNs).
8. Built In — "Collaborative Filtering: A Simple Introduction."
9. Turing.com — "A Guide to Content-based Filtering in Recommender Systems" and "How Collaborative Filtering Works in Recommender Systems."
10. Redis — "Collaborative filtering: How to build a recommender system" (hybrid recommender design, cold-start and sparsity).
11. CodSoft — Internship Offer Letter and Program Presentation ("Artificial Intelligence"), 2026.